COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Particular	Details
Department	Computer Science and Bio Science
Programme	B.Tech Computer Science and Bio Science
Course Code & Course Name	CSA0315 – Data Structures – Slot D
Academic Year / Batch	2026/2024
Faculty Name	Dr.T.Rajesh Kumar
Assignment Title	Design and Implementation of a High-Speed E-Commerce Product Search and Recommendation System Using Advanced Data Structures
Date of Issue	1/09/2026
Date of Submission	1/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO1 and CO2
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze
SDG Mapping	SDG 9 – Industry, Innovation and Infrastructure; SDG 11 – Sustainable Cities and Communities
Industry / Societal Relevance	High-speed product search systems are essential for modern e-commerce platforms. Efficient search and recommendation technologies improve customer experience, reduce search time, optimize computational resources, and support scalable digital commerce infrastructure.

B. Assignment Problem / Challenge

Modern e-commerce platforms manage millions of products containing information such as product names, product IDs, categories, keywords, brands, prices, and other searchable attributes. Customers expect search results to be returned quickly, even when using different types of queries.

The major challenge is that different search queries require different searching techniques. For example, an exact product ID search requires a different approach from prefix autocomplete or keyword searching.

The system must therefore:

- Support exact product searches.
- Support prefix-based searches such as “lap,” “laptop,” and “laptop stand.”

- Support keyword-based searching.
- Support category-based searching.
- Handle insertion, deletion, and updating of products.
- Maintain synchronization between multiple search indexes.
- Optimize frequently repeated queries.
- Control memory consumption.
- Compare different searching algorithms.
- Maintain a normal response time close to 100 milliseconds.
- Scale efficiently for increasingly large product datasets.

The engineering challenge is to determine which data structure should be used for each type of query and integrate them into a single efficient hybrid search system.

C. Problem Statement

A large e-commerce company requires a high-speed product search and recommendation system capable of managing information about millions of products.

Each product contains information such as:

- Product ID
- Product name
- Category
- Keywords
- Brand
- Price
- Popularity
- Other searchable attributes

Customers should be able to search using exact product names, product identifiers, categories, keywords, and partial words such as “lap,” “laptop,” and “laptop stand.”

The system must support both:

1. Exact-match searching
2. Prefix-based searching

while maintaining a normal response time of approximately 100 milliseconds.

To achieve this objective, appropriate data structures and algorithms such as arrays, linked lists, linear search, binary search, hash tables, tries, suffix arrays, and LCP arrays must be studied and applied.

The system must also efficiently support continuous:

- Product insertion
- Product deletion
- Product updating

while maintaining synchronization between search indexes and controlling memory consumption.

Frequently repeated queries should be optimized using suitable caching mechanisms.

The main objective is to evaluate the selected data structures and algorithms based on:

- Search speed
- Update efficiency
- Memory usage
- Scalability
- Support for exact searching
- Support for prefix searching

and design an efficient solution suitable for a large-scale e-commerce environment.

This formulation follows the SmartSearch problem statement in your existing assignment report.

D. Requirements and Constraints

D.1 Functional Requirements

The SmartSearch system shall:

1. Allow users to search products using an exact product ID.
2. Allow users to search using an exact product name.
3. Support prefix-based autocomplete.
4. Support keyword-based searching.
5. Support category-based searching.
6. Provide relevant product recommendations.
7. Support adding new products.
8. Support updating existing products.
9. Support deleting products.
10. Synchronize all search indexes after product modifications.
11. Detect duplicate product IDs.
12. Cache frequently repeated search queries.
13. Display search algorithm information and performance metrics.
14. Provide algorithm comparison and benchmarking functionality.

D.2 Performance Requirements

The system should:

- Maintain normal search response times close to **100 milliseconds**.
- Perform exact product lookup efficiently.

- Provide real-time prefix autocomplete.
- Improve repeated query performance through caching.
- Support benchmarking across multiple dataset sizes.
- Monitor search latency and cache performance.

D.3 Technical Requirements

The system uses:

- React
- TypeScript
- Vite
- Node.js
- HTML
- CSS
- npm

The major data structures and algorithms used are:

- Arrays
- Linear Search
- Binary Search
- Hash Tables
- Tries
- Inverted Indexes
- Category Indexes
- Doubly Linked Lists
- Hash Maps
- LRU Cache
- Suffix Arrays
- LCP Arrays
- Jaccard Similarity

D.4 Resource Constraints

The system must consider:

- Efficient memory usage.
- Limited cache capacity.
- Increasing dataset size.
- Browser-based execution limitations.
- Processing overhead during index rebuilding.
- Performance differences between algorithms.

D.5 Reliability Requirements

The system should ensure:

- Duplicate product IDs are prevented.
- Deleted products are removed from all indexes.
- Updated products are correctly re-indexed.
- Search indexes remain consistent.
- Cached results are invalidated after catalog modifications.

D.6 Scalability Requirements

The system should demonstrate how different algorithms behave as the dataset increases.

Benchmark datasets may include:

- 1,000 products
- 10,000 products
- 100,000 products
- 1,000,000 products

D.7 Sustainability and Societal Considerations

Efficient searching reduces unnecessary computational operations. Using appropriate data structures can reduce repeated scanning of large datasets and improve resource utilization.

The project is relevant to:

SDG 9 – Industry, Innovation and Infrastructure

The project demonstrates the use of efficient algorithms and scalable computing techniques for modern digital infrastructure.

SDG 11 – Sustainable Cities and Communities

Efficient digital commerce systems can support better access to products and services while reducing unnecessary computational resource consumption.

E. Student Work

1. Problem Understanding and Formulation

1. PROBLEM UNDERSTANDING AND FORMULATION

1.1 What is the Problem?

The problem is to design a high-speed product search and recommendation system for a large e-commerce environment.

A simple linear search through every product is inefficient when the number of products becomes very large. Different types of queries also require different searching mechanisms.

For example:

Query	Example	Suitable Data Structure
Exact ID	P1001	Hash Table
Exact Name	Laptop Stand	Hash Table
Prefix	lap	Trie
Keyword	gaming laptop	Inverted Index
Category	Electronics	Category Index
Repeated Query	laptop	LRU Cache

Therefore, the main problem is to develop a **hybrid search architecture** instead of using one algorithm for every search operation.

1.2 Expected Outcomes

The expected outcomes are:

- A functional product search system.
- Fast exact product searching.
- Real-time prefix autocomplete.
- Keyword and category searching.

- Efficient repeated query processing.
- Product CRUD functionality.
- Automatic index synchronization.
- Product recommendations.
- Algorithm comparison.
- Performance monitoring and analysis.

1.3 Available Information and Data

The system uses product information containing:

- Product ID
- Product name
- Category
- Keywords
- Brand
- Price
- Popularity
- Searchable attributes

A realistic dataset of products is used for implementing and testing the search engine.

1.4 Assumptions

The following assumptions are made:

1. Every product has a unique product ID.
2. Product names and keywords can be normalized before indexing.
3. Frequently repeated queries benefit from caching.
4. Product updates must be reflected across all indexes.
5. The production search system can use multiple data structures simultaneously.
6. Search performance depends on query type and dataset size.

1.5 Constraints

The proposed solution must satisfy:

- Efficient search performance.
- Approximate 100 ms response target.

- Limited memory usage.
- Correct CRUD synchronization.
- Efficient repeated query processing.
- Scalability for larger datasets.
- Reliable search index consistency.

---

2. APPLICATION OF COURSE KNOWLEDGE

This project applies several important Data Structures concepts.

2.1 Arrays

Arrays are used for:

- Product storage.
- Sequential processing.
- Linear search.
- Algorithm benchmarking.

Complexity

- Access: **O(1)**
- Linear search: **O(n)**

2.2 Linear Search

Linear Search checks products sequentially.

Complexity

- Best Case: **O(1)**
- Average Case: **O(n)**
- Worst Case: **O(n)**

It is useful as a baseline for comparing other searching algorithms.

2.3 Binary Search

Binary Search is performed on sorted data.

Complexity

- Searching: **O(log n)**
- Sorting/Preprocessing: **O(n log n)**

Binary Search is significantly more scalable than Linear Search for sorted datasets.

2.4 Hash Table

Hash Tables are used for:

- Exact product ID lookup.
- Exact product name lookup.
- Fast duplicate detection.

Complexity

Average lookup:

O(1)

2.5 Trie

A Trie is used for prefix searching.

Example:

l

↓

la

↓

lap

↓

lapt

↓

lapto

↓

laptop

Complexity

O(L)

where **L** is the length of the search prefix.

2.6 Linked List and LRU Cache

A Doubly Linked List is combined with a Hash Map to implement the LRU Cache.

This allows:

- Fast cache lookup.

- Fast insertion.
- Fast deletion.
- Identification of the least recently used query.

Complexity

- Lookup: **O(1)**
- Insertion: **O(1)**
- Deletion: **O(1)**

2.7 Inverted Index

The Inverted Index maps keywords to products.

Example:

gaming

↓

Product IDs

laptop

↓

Product IDs

This avoids scanning every product for every keyword search.

2.8 Suffix Array and LCP Array

Suffix Arrays and LCP Arrays are implemented in the Algorithm Lab for studying substring searching and string processing.

The LCP Array is generated using the **Kasai algorithm**, which operates in linear time after suffix array construction.

2.9 Recommendation Algorithm

The recommendation system uses weighted similarity.

Recommendation Formula

Recommendation Score =

(Category Score × 40%)

+

(Keyword Similarity × 25%)

+

(Attribute Similarity × 20%)

+

(Brand Match × 5%)

+

(Popularity Score × 10%)

Keyword similarity uses the **Jaccard Similarity Index**:

Jaccard Similarity =

Intersection of Sets

———————————

Union of Sets

---

3. SOLUTION / DESIGN / METHODOLOGY

3.1 Proposed Solution

The proposed solution is called **SmartSearch**.

SmartSearch uses a **Hybrid Search Architecture**.

Instead of performing every search using the same algorithm, the Query Router identifies the query type and selects the most suitable data structure.

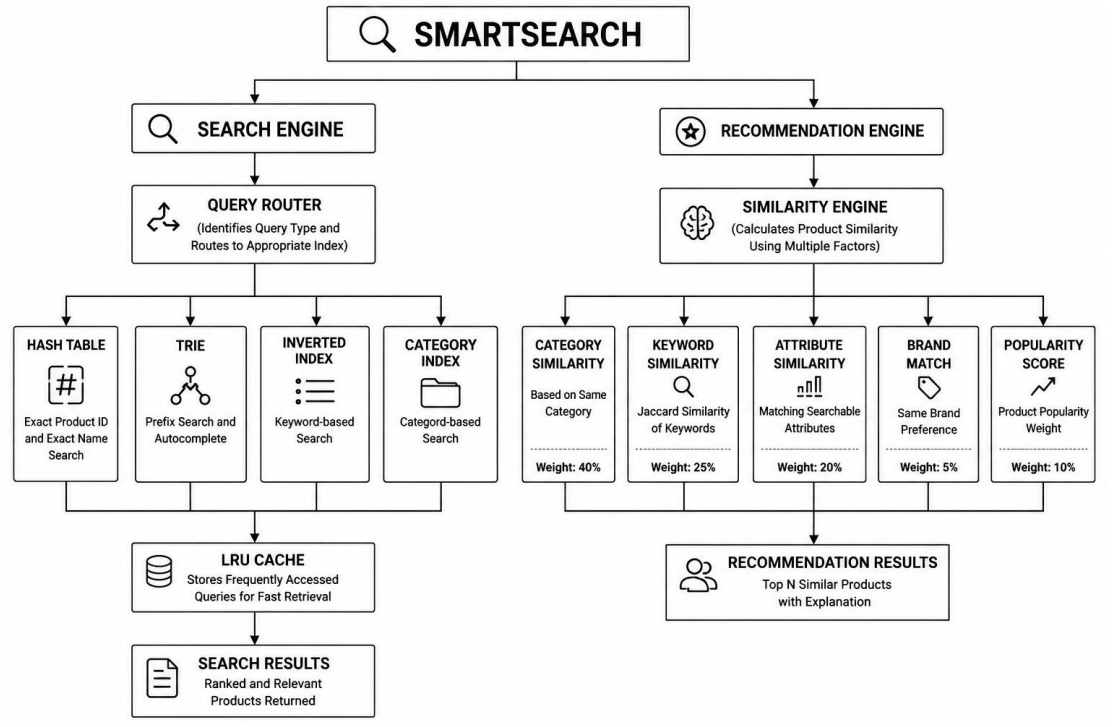
3.2 System Architecture

Use your existing **black-and-white university-style architecture diagram** here.

Figure Caption:

Figure 1: SmartSearch Hybrid Search and Recommendation System Architecture

The architecture contains:



3.3 Search Methodology

Step 1: User Enters a Query

Example:

lap

Step 2: Normalize Query

The query is converted into a standard format.

Step 3: Check Cache

The system checks whether the query already exists in the LRU Cache.

Step 4: Query Classification

The Query Router determines whether the query is:

- Product ID
- Exact name
- Prefix
- Keyword
- Category

Step 5: Select Appropriate Data Structure

Query Type	Data Structure
Exact ID	Hash Table
Exact Name	Hash Table
Prefix	Trie
Keyword	Inverted Index
Category	Category Index

Step 6: Return Search Results

The matching products are returned to the user.

Step 7: Store Result in Cache

The result is stored in the LRU Cache for faster repeated searching.

3.4 CRUD Methodology

The **IndexManager** manages:

- Add Product
- Update Product
- Delete Product

Whenever a product changes, the IndexManager synchronizes:

- Hash Table
- Trie
- Inverted Index
- Category Index

The LRU Cache is invalidated after product modifications.

---

4. USE OF MODERN TOOLS

The following modern tools were used in the implementation:

Tool	Purpose
React	User interface development
TypeScript	Application programming
Vite	Development and production build
Node.js	Runtime environment
npm	Package management

Visual Studio Code	Source code development
Git/GitHub	Version control
Chrome/Modern Browser	Application execution and testing

Evidence of Tool Usage

Include screenshots showing:

1. Visual Studio Code project structure.
2. Running SmartSearch application.
3. Search functionality.
4. Algorithm Lab.
5. Performance Dashboard.
6. Admin Dashboard.
7. Terminal build output.

Figure Caption:

Figure 2: SmartSearch Development Environment and Project Implementation

---

5. RESULTS AND VALIDATION

5.1 Test Cases

Test Case 1: Exact Product ID Search

Parameter	**Result**
Input	P1001
Expected Result	Correct product returned
Actual Result	Correct product returned
Status	PASS

Test Case 2: Prefix Search

Parameter	**Result**
Input	lap
Expected Result	Matching prefix products displayed

Actual Result	Matching products displayed
Status	PASS

Test Case 3: Keyword Search

Parameter	**Result**
Input	gaming laptop
Expected Result	Relevant products returned
Actual Result	Relevant products displayed
Status	PASS

Test Case 4: LRU Cache

Parameter	**Result**
Input	Same query twice
Expected Result	First search MISS, second search HIT
Actual Result	Cache MISS followed by Cache HIT
Status	PASS

Test Case 5: Add Product

Parameter	**Result**
Input	New Product
Expected Result	Product added to all indexes
Actual Result	Successfully synchronized
Status	PASS

Test Case 6: Duplicate Product ID

Parameter	**Result**
Input	Existing Product ID
Expected Result	Duplicate rejected

Actual Result	Duplicate prevented
Status	PASS

Test Case 7: Update Product

Parameter	**Result**
Input	Modified Product
Expected Result	Old index entries removed and new entries added
Actual Result	Successfully synchronized
Status	PASS

Test Case 8: Delete Product

Parameter	**Result**
Input	Existing Product
Expected Result	Product removed from all indexes
Actual Result	Successfully removed
Status	PASS

Test Case 9: Recommendation Engine

Parameter	**Result**
Input	Selected Product
Expected Result	Similar products recommended
Actual Result	Relevant recommendations generated
Status	PASS

Test Case 10: Algorithm Benchmarking

Parameter	**Result**
Input	Increasing dataset sizes

Expected Result	Performance comparison generated
Actual Result	Benchmark results generated
Status	PASS

5.2 Build Validation

The application should be validated using:

npm run build

The build process checks:

- TypeScript compilation.
- Module dependencies.
- Application bundling.
- Production readiness.

5.3 Test Suite Validation

The following areas were tested:

- Core Data Structures
- CRUD Synchronization
- Recommendation Engine
- Algorithm Lab
- Performance Telemetry

---

6. ANALYSIS AND ENGINEERING DECISION

6.1 Alternative Solutions Considered

Alternative 1: Linear Search for All Queries

Advantages:

- Simple implementation.
- No additional indexing.

Disadvantages:

- O(n) search complexity.
- Poor scalability.
- Inefficient for millions of products.

Alternative 2: Binary Search

Advantages:

- O(log n) searching.
- Faster than Linear Search.

Disadvantages:

- Requires sorted data.
- Maintaining sorted data during continuous updates has overhead.
- Does not efficiently support autocomplete.

Alternative 3: Hash Table Only

Advantages:

- Very fast exact searching.
- Average O(1) lookup.

Disadvantages:

- Not suitable for prefix searching.
- Not suitable for autocomplete.

Alternative 4: Trie Only

Advantages:

- Excellent prefix searching.
- Suitable for autocomplete.

Disadvantages:

- Can consume significant memory.
- Not ideal as the only structure for all query types.

6.2 Final Engineering Decision

The final solution selected was a **Hybrid Search Architecture**.

The reason is that no single data structure efficiently solves every search requirement.

The selected design combines:

- Hash Table → Exact Search
- Trie → Prefix Search

- Inverted Index → Keyword Search
- Category Index → Category Search
- LRU Cache → Repeated Queries

This provides better flexibility and performance.

6.3 Complexity Comparison

Algorithm / Structure	Main Operation	Complexity
Linear Search	Search	O(n)
Binary Search	Search	O(log n)
Hash Table	Exact Lookup	O(1) average
Trie	Prefix Search	O(L)
LRU Cache	Lookup	O(1)
Inverted Index	Keyword Retrieval	Depends on postings

6.4 Key Finding

The major engineering finding is:

A hybrid architecture using multiple specialized data structures provides better performance and flexibility than using one searching algorithm for every query.

6.5 Limitations

The current implementation has some limitations:

- Browser memory limits affect extremely large datasets.
- Benchmark results depend on hardware.
- In-memory indexes may become large.
- Large-scale production systems would require database and distributed infrastructure.
- Recommendation calculations can become expensive for very large catalogs.

---

7. BROADER CONSIDERATIONS

7.1 Sustainability

Efficient algorithms reduce unnecessary computations.

For example, using a Hash Table instead of repeatedly scanning an entire dataset can reduce computational overhead.

Caching repeated queries also avoids unnecessary repeated processing.

7.2 Societal Impact

Fast product search improves:

- User experience.
- Accessibility to products.
- Digital commerce efficiency.

7.3 Accessibility

The user interface should provide:

- Clear search functionality.
- Keyboard navigation.
- Readable text.
- Clear result indicators.
- Meaningful feedback for empty results.

7.4 Privacy and Security

A production version should consider:

- User search history privacy.
- Secure product administration.
- Input validation.
- Protection against malicious search requests.

The current academic implementation focuses primarily on Data Structures and algorithmic performance.

7.5 Professional Responsibility

Performance results should be reported honestly.

Measured performance values may vary depending on:

- Hardware.
- Browser.

- Dataset size.
- System load.

Therefore, benchmark results should not be falsely represented as fixed universal values.

---

8. CONCLUSION

The SmartSearch project successfully demonstrates the design and implementation of a high-speed e-commerce product search and recommendation system using advanced data structures and algorithms.

The final solution uses a hybrid approach in which different queries are processed using the most appropriate data structure.

- Hash Tables are used for exact searching.
- Tries are used for prefix autocomplete.
- Inverted Indexes are used for keyword searching.
- Category Indexes are used for category retrieval.
- LRU Caching improves repeated query performance.

The system also supports insertion, deletion, and updating of products through a centralized IndexManager that maintains synchronization across all search indexes.

The project further demonstrates the comparison of Linear Search, Binary Search, Hash Tables, Tries, Suffix Arrays, and LCP Arrays through an Algorithm Lab and benchmarking environment.

The major conclusion is that a **hybrid combination of data structures is more effective than relying on a single search technique** for a large-scale e-commerce environment.

The project successfully achieves its objective of demonstrating an efficient, scalable, and intelligent product search architecture.

---

9. STUDENT REFLECTION

What We Learned

This assignment provided practical experience beyond theoretical Data Structures concepts.

We learned that understanding the time complexity of an algorithm is important, but selecting the correct data structure based on the actual problem is equally important.

Through this project, we gained practical understanding of:

- Hash Tables
- Tries
- Linked Lists
- Caching

- Linear Search
- Binary Search
- Indexing
- Algorithm benchmarking
- CRUD synchronization
- Scalability analysis

We also learned how multiple data structures can work together in a single software system.

Challenges Faced

The major challenges included:

- Maintaining synchronization between multiple indexes.
- Handling product updates correctly.
- Preventing duplicate IDs.
- Implementing efficient cache invalidation.
- Comparing algorithms fairly.
- Managing large benchmark datasets.
- Integrating Data Structures with a complete web application.

Improvements with Additional Time and Resources

With additional time and resources, the following improvements could be implemented:

1. Database integration.
2. Distributed search indexing.
3. More advanced ranking algorithms.
4. Machine learning-based recommendations.
5. Persistent caching.
6. Larger real-world product datasets.
7. User personalization.
8. Advanced typo correction and fuzzy searching.
9. Cloud deployment.
10. Multi-server scalability testing.

---

10. REFERENCES

Use the following **APA 7th edition references**:

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). The MIT Press.
2. Weiss, M. A. (2014). *Data structures and algorithm analysis in C++* (4th ed.). Pearson.
3. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.
4. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data structures and algorithms in Java* (6th ed.). Wiley.
5. Knuth, D. E. (1998). *The art of computer programming, volume 3: Sorting and searching* (2nd ed.). Addison-Wesley.
6. Kasai, T., Lee, G., Arimura, H., Arikawa, S., & Park, K. (2001). Linear-time longest-common-prefix computation in suffix arrays and its applications. In A. Amir and G. M. Landau (Eds.), *Combinatorial pattern matching* (pp. 181–192). Springer.
7. Meta Platforms, Inc. (n.d.). *React documentation*. Meta Platforms, Inc.
8. Microsoft. (n.d.). *TypeScript documentation*. Microsoft.
9. Vite. (n.d.). *Vite documentation*. Vite.
10. Mozilla. (n.d.). *MDN Web Docs: JavaScript guide*. Mozilla Foundation.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	**100**

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation			L3/L4	10
Application of knowledge			L3/L4	20
Solution / Design / Implementation			L4/L5/L6	20
Modern tool usage			L3/L4	10
Validation			L4/L5	15
Analysis & justification			L4/L5	15
Broader considerations			L4/L5	5
Documentation & reflection			L3/L4	5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering:

Documents to be Maintained

The department/course faculty shall maintain:

1. Assignment question/problem statement
2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable
6. Evaluated scripts/reports
7. Marks and rubric-wise assessment
8. CO attainment analysis
9. Sample student work representing different performance levels
10. Faculty analysis and continuous-improvement action